

MiniTask: Designing and Evaluating a Modern C++20 Concurrent Task Runtime

A lightweight C++20 task runtime with bounded queues, per worker priorities, configurable work stealing, and low latency urgent task handling



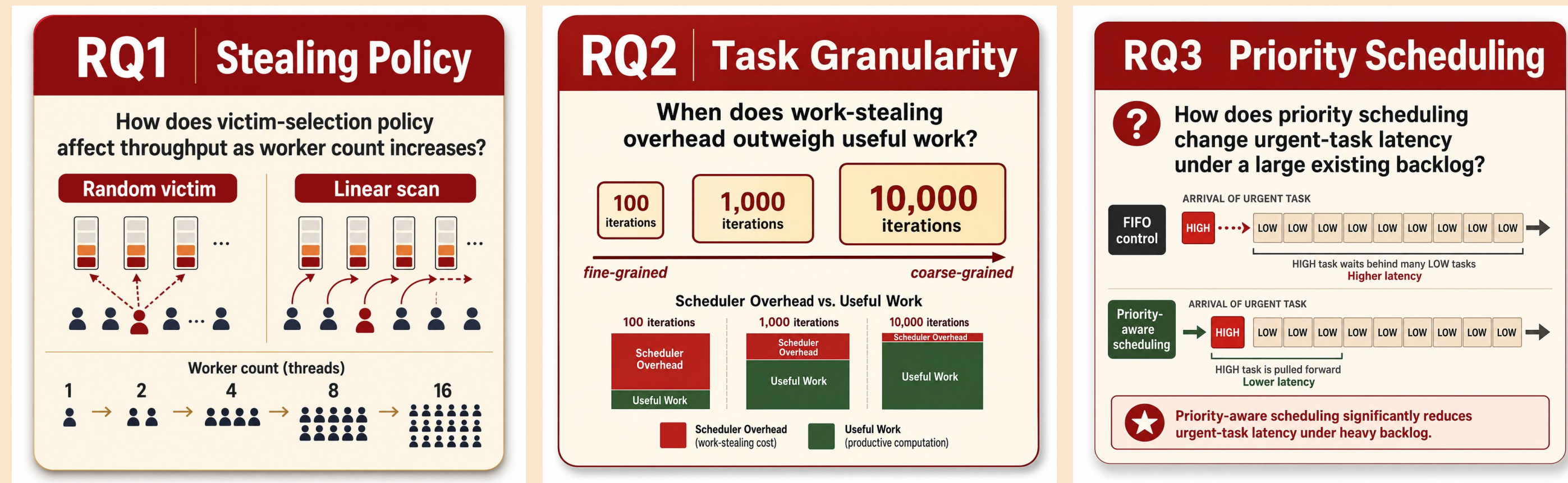
Darsh Rank, Master's in Computer Science
North Carolina State University
drank@ncsu.edu, +1 9843799309

NC STATE
UNIVERSITY

Summary and Motivation

MiniTask is a C++20 thread pool with per-worker bounded HIGH/LOW queues, configurable work stealing, and a burst limit that prevents LOW-task starvation. The design isolates steal policy, task granularity, and priority so each can be measured independently.

- For fine-grained tasks, scheduling overhead becomes comparable to the useful work itself.
- Per-worker queues create load imbalance when some workers drain faster than others.
- Urgent tasks must stay responsive even when thousands of normal tasks are already queued.



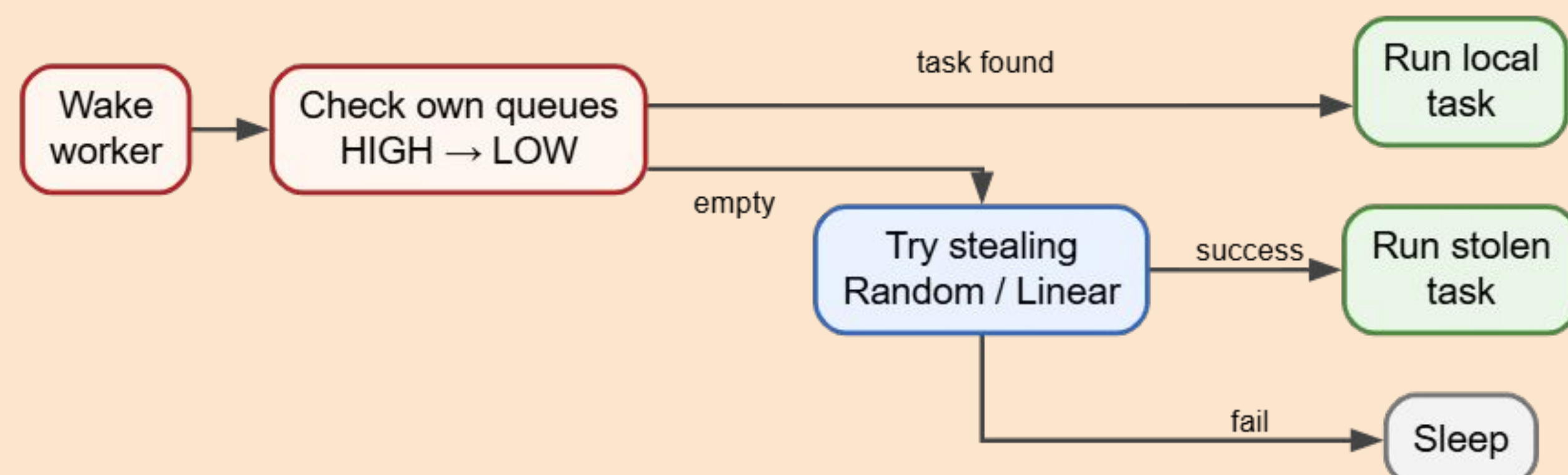
MiniTask C++20 API

```
ThreadPool pool(8, 100'000, StealPolicy::RANDOM);
auto future = pool.submit( TaskPriority::HIGH, [] { return compute(); });

pool.submitDetached( TaskPriority::LOW, [] { background_work(); });

auto result = future.get();
pool.wait();
std::cout << "submitted: " << pool.submittedTasks()
          << " completed: " << pool.completedTasks()
          << " queued: " << pool.queuedTasks();
```

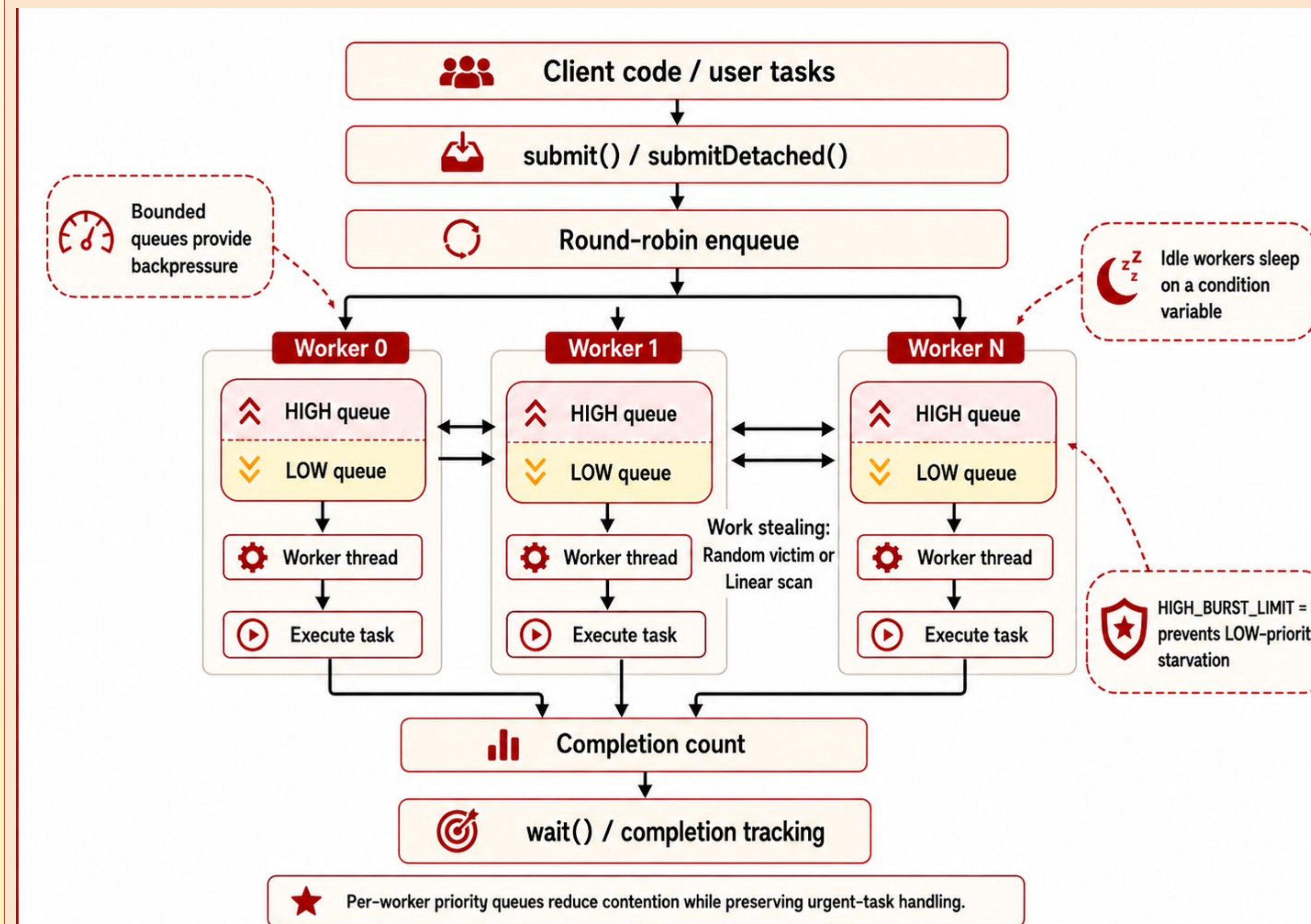
Worker Scheduling Logic



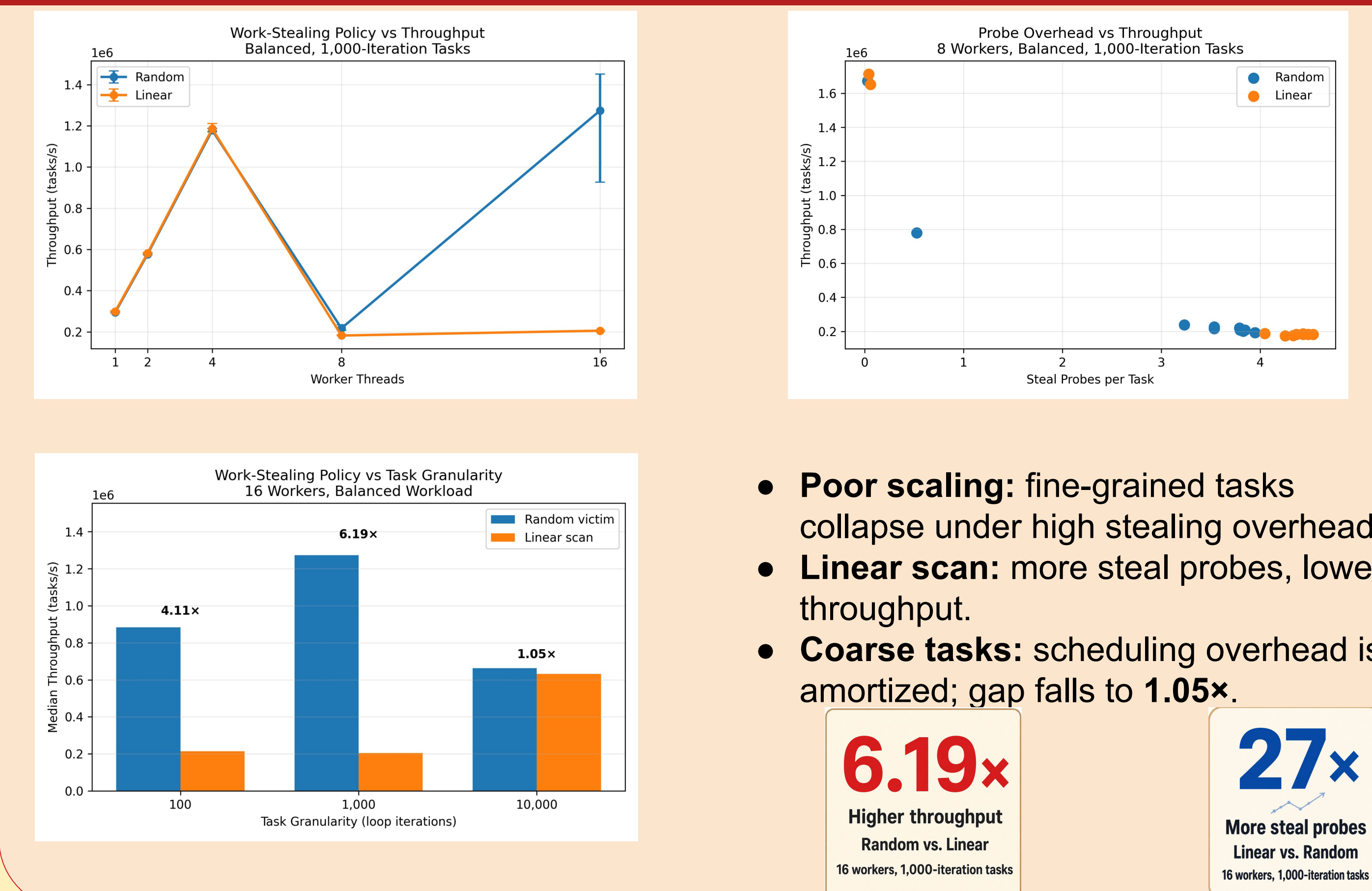
- Own queue first**: workers prefer local HIGH/LOW tasks before attempting a steal.
- Priority fairness**: after `HIGH_BURST_LIMIT = 3` consecutive HIGH tasks, LOW work is serviced if available.
- Idle efficiency**: if no local or stolen task is found, the worker sleeps on a condition variable instead of busy-waiting.

Steal policy is configurable: random victim or linear scan

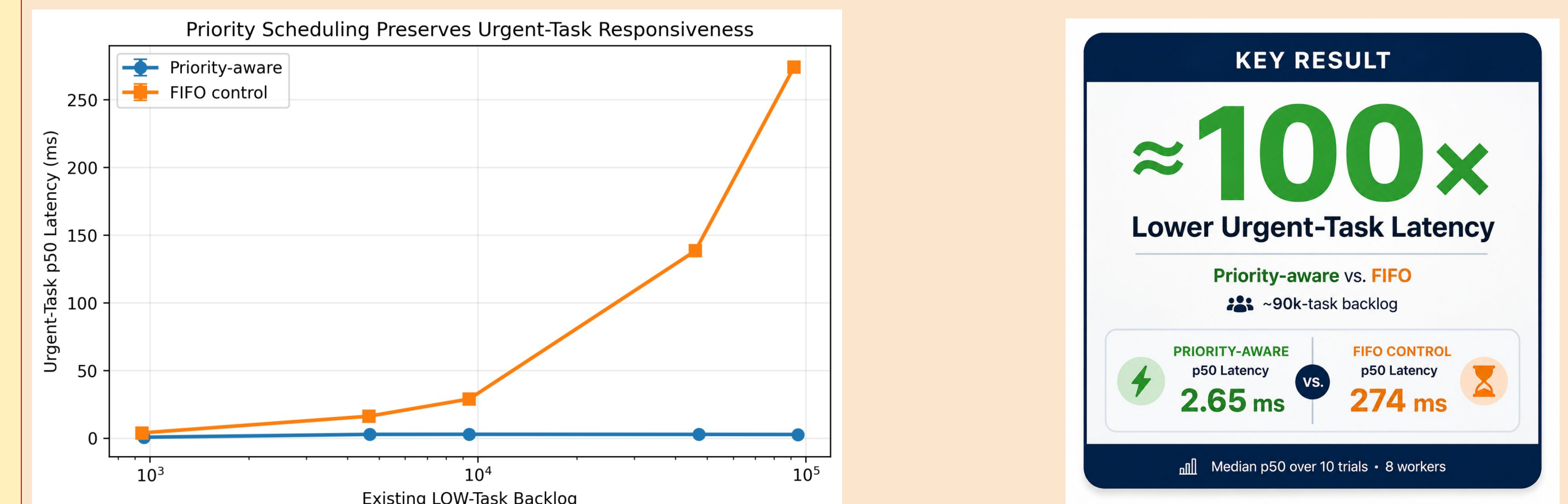
Runtime Architecture



Work-Stealing Results



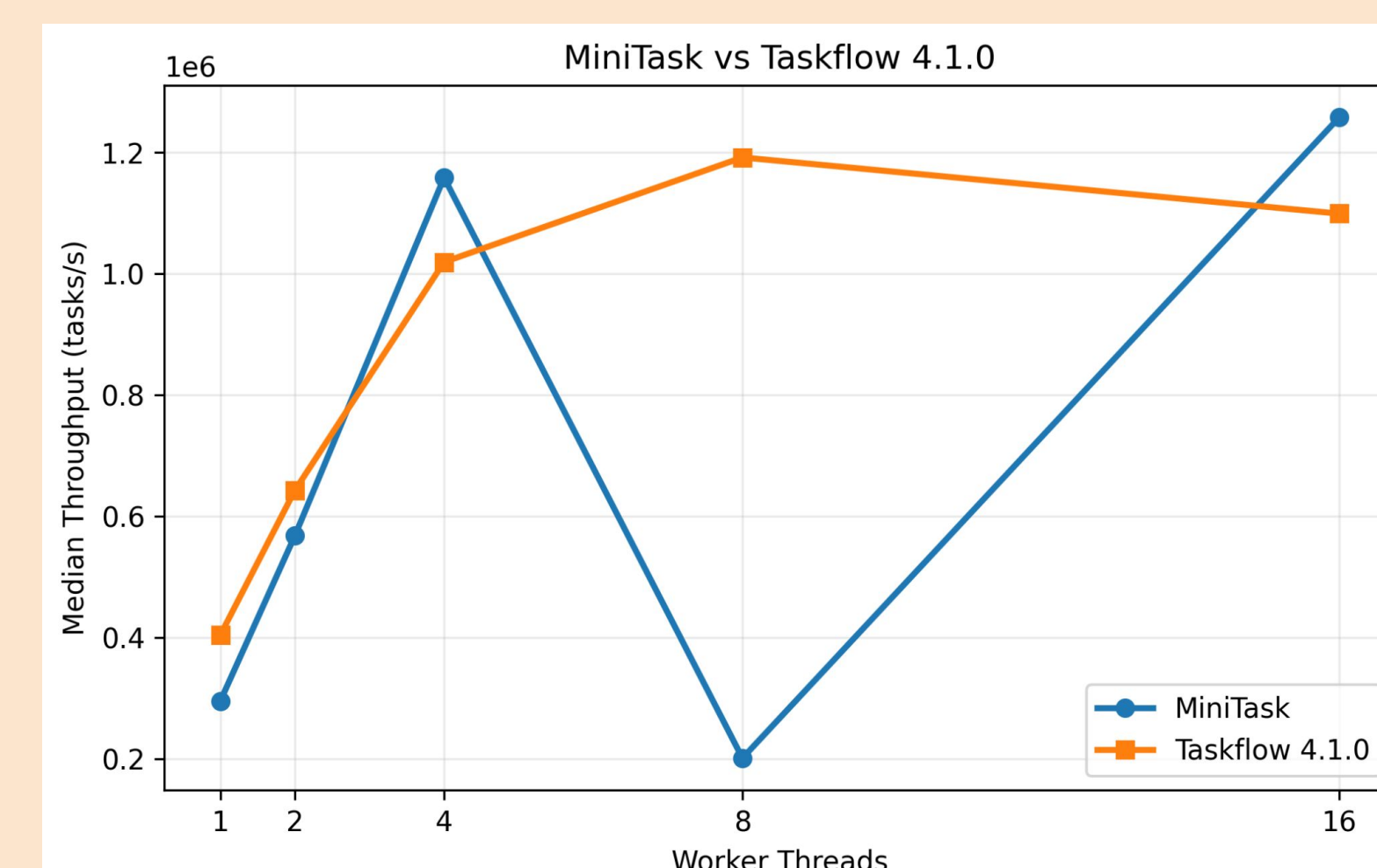
Priority Scheduling Results



Experimental Setup

Platform	NCSU VCL virtual machine
CPU	16 vCPUs, Intel Xeon Skylake-class (KVM)
Memory	125 GiB RAM
OS	Ubuntu 22.04.5 LTS, Linux 5.15
Compiler	GCC 11.4.0, -O2, C++20, -pthread
Worker counts	1, 2, 4, 8, 16
Stealing policies	Random victim / Linear scan
Repetitions	10 trials per configuration

MiniTask vs Taskflow



- Competitive outside the pathological regime**: MiniTask matches or exceeds Taskflow at 4 and 16 workers.
- 8-worker collapse**: MiniTask falls to 201k tasks/s while Taskflow sustains 1.19M tasks/s.

1M balanced tasks · 1,000 iterations/task · median of 10 trials · Taskflow 4.1.0

Conclusions & Future Work

- Steal policy matters most for fine-grained work**: excessive probing can collapse throughput; random victim selection reached up to **6.19x** higher median throughput than linear scan.
- Priority scheduling preserves responsiveness**: urgent-task p50 latency stayed near **2-3 ms** even with ~90k queued LOW tasks, versus **274 ms** under FIFO.
- Future work**: compare against additional runtimes, explore adaptive/backoff stealing policies, and evaluate under CPU pinning, NUMA, and realistic workloads.



GitHub: Code & Benchmarks



LinkedIn: Let's Connect